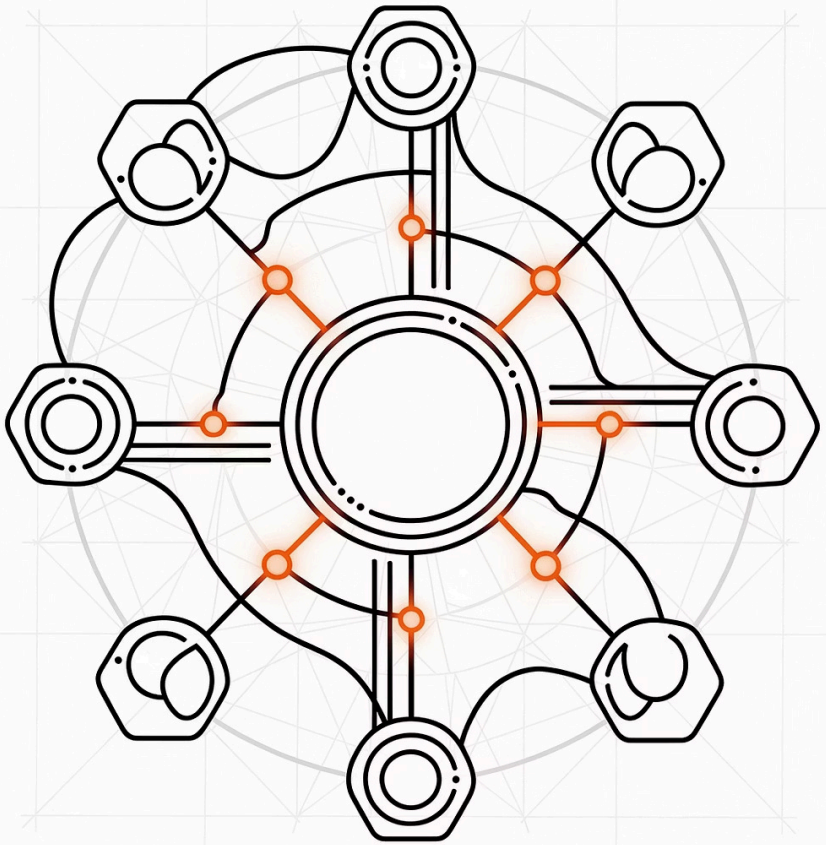


Module 4: Agents on Amazon Bedrock

Research Findings — Current State, February 2026

AMAZON BEDROCK AGENTS

MODULE 4

Amazon Bedrock Agents Overview

Amazon Bedrock Agents enable developers to build intelligent, autonomous agents that go far beyond simple question-answering. These agents understand natural language instructions, decompose complex goals into actionable steps, and call external systems to get work done.

Natural Language

Understand and interpret complex user instructions without rigid command structures.

Task Decomposition

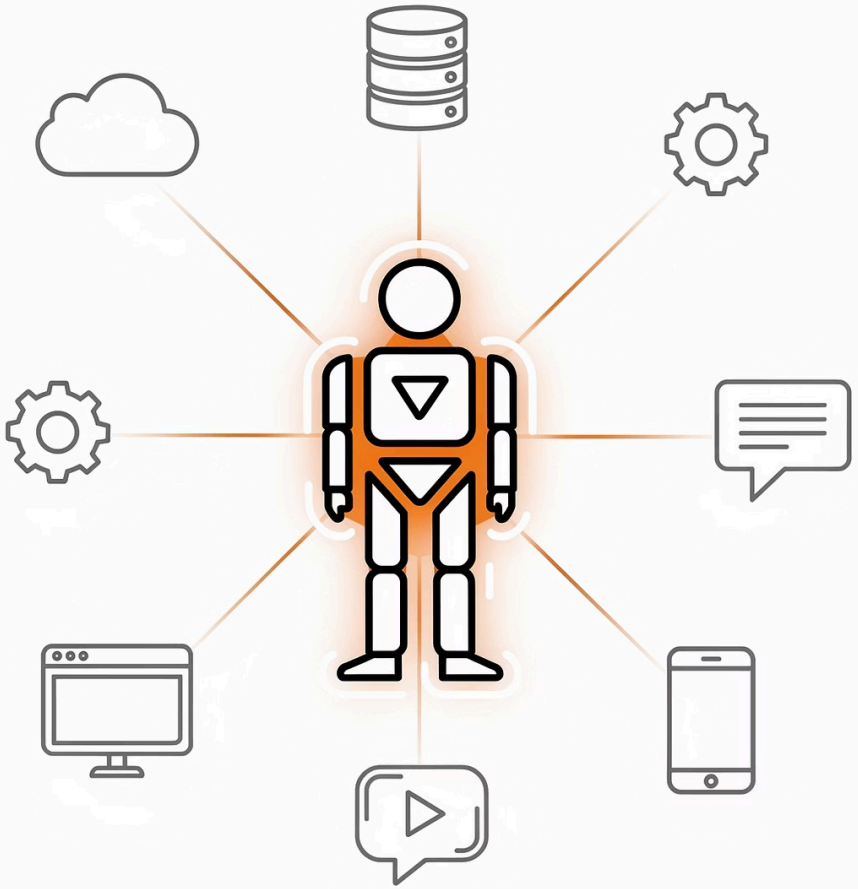
Break multi-step goals into discrete, executable sub-tasks with reasoning.

Tool & API Calls

Invoke external APIs, Lambda functions, and MCP-connected services on demand.

Grounded Answers

Use Knowledge Bases for retrieval-augmented generation and factually grounded responses.



Agent Architecture Components

Every Amazon Bedrock Agent is assembled from four core components. Understanding each layer is essential for designing reliable, production-ready agentic systems.

1

Agent Instructions

Define the agent's role, scope, and behavioral constraints. Be specific about purpose, define boundaries clearly, include edge-case examples, and specify tone, style, and safety guardrails.

2

Action Groups

Collections of callable actions backed by OpenAPI 3.0 schemas or function definitions, executed via Lambda or HTTP endpoints. Natural-language descriptions guide model selection. Refer to the Bedrock quotas page for current per-region limits.

3

Knowledge Bases

Attach one or more Bedrock Knowledge Bases for RAG. Answer questions from enterprise documents, provide grounded context, and combine retrieval with live actions like ticket creation.

4

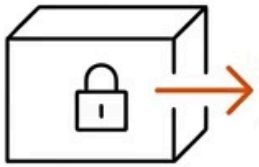
Foundation Model

Choose the FM driving agent reasoning and planning. Supported models include Amazon Nova, Anthropic Claude, Meta Llama, and GPT-OSS. Key trade-offs: reasoning quality, latency, multimodal capability, and context window size.

Amazon Bedrock AgentCore (2026)

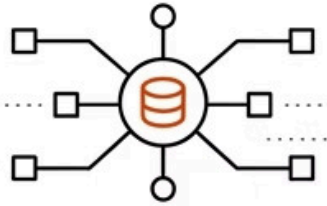
AgentCore is AWS's production-grade suite of capabilities for building, operating, and scaling AI agents. It addresses the hardest infrastructure challenges — runtime reliability, tool integration, identity, memory, and observability — so engineering teams can focus on agent logic.

Runtime



Secure execution.

Gateway

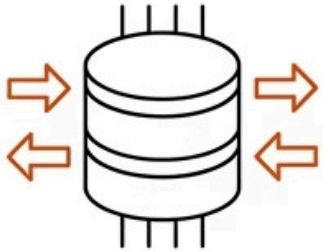


Centralized registry.

Identity

User auth.

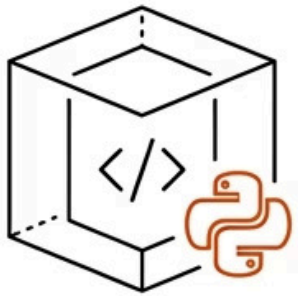
Memory



Session context.

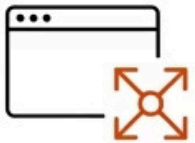
Code Interpreter

Sandboxed.



Python sandbox.

Browser Tool



Web content.

Observability

Traces & metrics.

AgentCore Runtime

Enterprise-grade, secure, scalable runtime for production agents. Features automatic load-based scaling, session isolation across users and conversations, and deep integration with Bedrock service tiers and quotas.

AgentCore Gateway

Centralized tool and API gateway with first-class MCP support. Unites multiple MCP servers behind a single endpoint, translating them into Bedrock-compatible tools. Handles auth, throttling, versioning, and cross-agent tool reuse — reducing M×N integration complexity to M+N.

AgentCore Identity

Propagates user identity from the calling application all the way into agent tool calls. Enables fine-grained, per-user or per-tenant access control, session-level permission scoping, and full audit trails for compliance.

AgentCore Memory

Gives agents persistent context beyond a single session. Stores conversation history, user preferences, and past task outcomes. Enables personalization and continuity across channels — essential for customer support, sales, and productivity copilots.

AgentCore Code Interpreter

Sandboxed Python execution for data analysis, CSV/JSON transformation, and chart generation. Results are returned to the client or saved to S3 — enabling numeric reasoning inside live agent workflows.

AgentCore Browser Tool & Observability

Browser Tool

Enables agents to navigate web pages, retrieve HTML/text content, and extract information from documentation or knowledge sites that are not indexed in a Knowledge Base. Supports research, data gathering, and live doc reading. Not a full RPA system — limited automation patterns apply.

Observability

Provides step-by-step execution traces for every agent invocation — tool calls with inputs, outputs, and timing; decision-point reasoning; latency breakdowns; and token usage. Integrates with CloudWatch for production monitoring, alerting, and debugging.

Agent Development Approaches

Bedrock Agents support three complementary action-definition strategies. Choose the one that best matches your team's existing contracts and tooling.

1

OpenAPI-Based Actions

Define actions via OpenAPI 3.0 schemas describing REST-style APIs. Best for teams with existing REST APIs, complex request/response structures, or established OpenAPI contracts.

2

Function-Based Actions

Define actions around functions with structured parameters and return types. Ideal for simple utilities, direct LLM-to-function mapping, or rapid prototyping without OpenAPI overhead. Implement logic in Lambda or your own service.

3

Model Context Protocol (MCP)

Standardized protocol that acts as a universal translator for tools and data sources. MCP servers register with AgentCore Gateway, which exposes them as action groups. Scales M×N integrations to M+N — composable, secure, and reusable across agents and organizations.

When using MCP, AgentCore Gateway translates MCP server capabilities into Bedrock-compatible tools automatically — no per-agent wiring required.

Lambda Integration: Best Practices

AWS Lambda is the primary backend for Bedrock Agent action groups. A clean Lambda integration requires the right IAM setup, predictable input/output formatting, and disciplined function design.

1 Keep Functions Focused

One clear responsibility per action. Avoid "god functions" that do too much — they're harder to debug and reason about.

2 Validate All Inputs

Treat agent-provided parameters as untrusted. Validate types, ranges, and required fields before executing any logic.

3 Handle Errors Gracefully

Return structured, meaningful error messages the agent can interpret and relay — not raw stack traces.

4 Set Timeouts & Log Everything

Use conservative Lambda timeouts. Log inputs, outputs, and context for debugging and auditing — scrubbing sensitive data as required.

Conceptual Lambda Handler

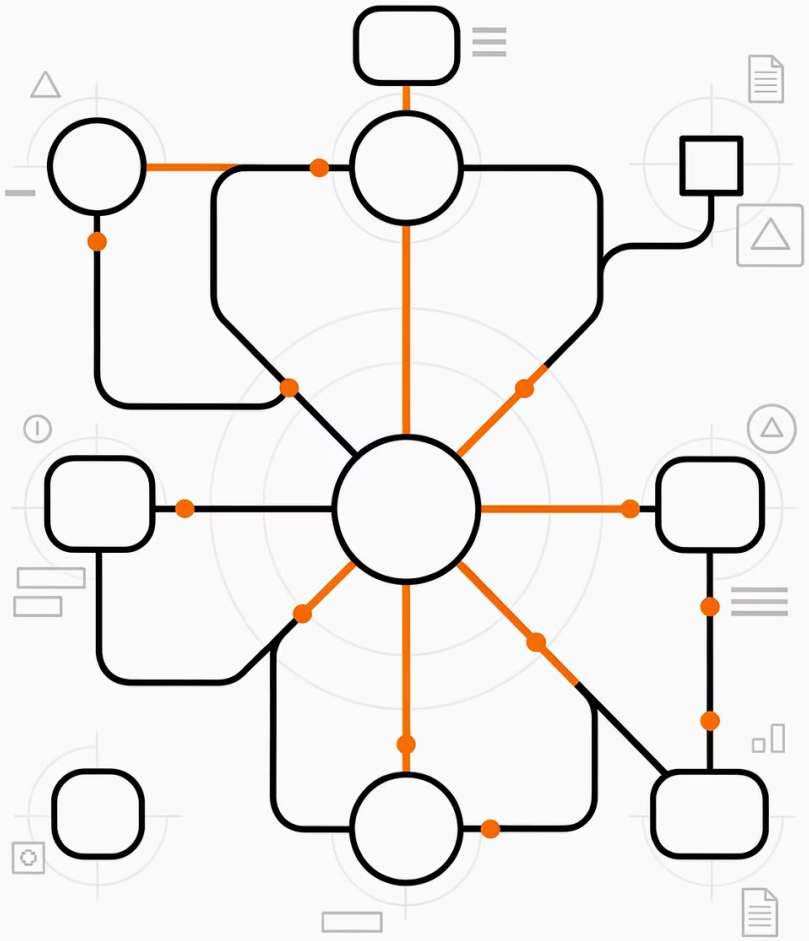
```
import json

def lambda_handler(event, context):
    action_group = event.get("actionGroup")
    function     = event.get("function")
    parameters   = event.get("parameters", {})

    # Dispatch to internal logic
    result = execute_action(function, parameters)

    return {
        "response": {
            "actionGroup": action_group,
            "function": function,
            "functionResponse": {
                "responseBody": {
                    "TEXT": {
                        "body": json.dumps(result)
                    }
                }
            }
        }
    }
```

This pattern matches the Bedrock Agents contract for returning tool results from a Lambda action group backend.



Multi-Agent Architectures

As workloads grow in complexity, a single agent is rarely enough. Multi-agent architectures distribute responsibility, improve maintainability, and allow specialized agents to excel at narrow tasks.

Sequential

Agents work in a pipeline — output of one feeds the next. Example: Research → Analysis → Report generation.

Parallel

Multiple agents work simultaneously on different sub-tasks. Results are aggregated at the end — ideal for multi-source data gathering.

Hierarchical

A supervisor agent delegates tasks to sub-agents based on type and complexity. Classic pattern: customer service router dispatching to billing, tech, or sales agents.

Collaborative

Agents share a common context and contribute partial solutions. Best for complex problems requiring negotiation or complementary expertise.

Agent Design Best Practices (2026)

01

Start Small, Define Clearly

Begin with a single, well-scoped use case. Expand capabilities incrementally once the core loop is stable and validated.

02

Instrument from Day One

Enable AgentCore Observability and CloudWatch metrics from the first deployment. Log tool invocations, token usage, latencies, and errors before you need to debug them.

03

Build a Deliberate Tooling Strategy

Design tools for agent consumption: clear names, unambiguous descriptions, and well-typed schemas. Validate inputs and handle errors predictably.

04

Automate Evaluation

Create test datasets and regression scripts. Continuously monitor quality and run A/B tests on prompts, tools, and model choices.

05

Decompose Complexity

Break complex workflows into smaller steps or separate agents with clear interfaces between them.

06

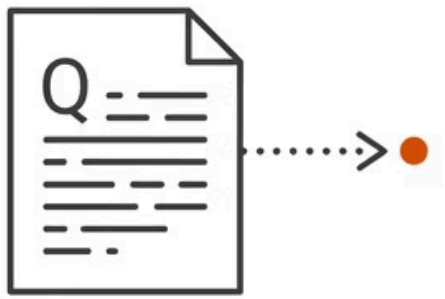
Combine Agents with Deterministic Code

Use traditional code for rules-based tasks; use agents for reasoning and ambiguous decisions. Hybrid designs maximize reliability and explainability.

Agents vs. RAG: When to Choose

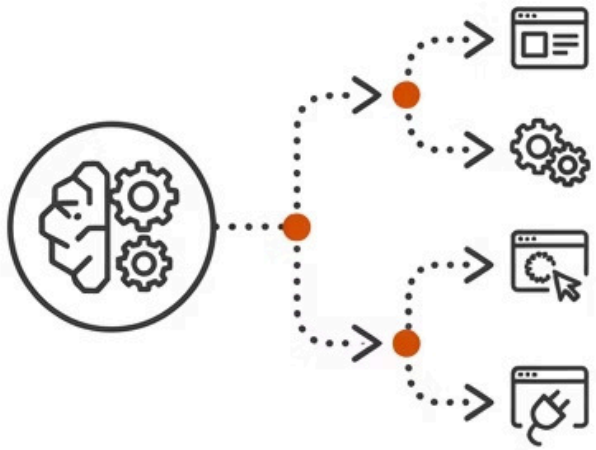
Both Agents and Knowledge Bases are powerful — but they serve different primary needs. The right choice depends on whether your workload is fundamentally about **information retrieval** or **task execution**.

RAG ONLY



Grounded Q&A. Deterministic Retrieval.

AGENTS ONLY



Task Orchestration. Multi-step workflows. API Integration.

HYBRID AGENTS + KB

Grounded Actions. Customer Service / Orders.

Hybrid pattern: Attach a Knowledge Base directly to an agent to combine RAG grounding with tool-based action — for example, reading return policy documents while simultaneously modifying an order record.

Agent Deployment & Versioning

Bedrock Agents use a **draft → version → alias** promotion model that enables safe, zero-downtime deployments, easy rollbacks, and A/B testing across production traffic.

Develop

Work in a draft for changes

Test

Validate with staging inputs

Version

Create immutable numbered version

Alias

Create or update alias to version

Versions

Numbered versions are immutable snapshots of the agent configuration. The working draft is always editable; versions are only created when you are ready to promote.

Aliases

Stable named pointers to specific versions. Switch versions by updating the alias — no application code changes required. Aliases support two states: `ACCEPT_INVOCATIONS` (active) and `REJECT_INVOCATIONS` (paused). Use multiple aliases for A/B testing or canary deployments.

Monitoring & Debugging

Robust observability is non-negotiable for production agents. Bedrock exposes native CloudWatch metrics alongside AgentCore's rich execution traces.

CloudWatch Metrics

InvocationCount / TotalTime

Total invocations and end-to-end wall-clock duration per agent call.

TTFT / ModelLatency

Time to first token and time spent inside underlying model calls — key latency indicators.

Token Usage & Throttles

InputTokenCount, OutputTokenCount, and throttled invocation counts for cost and capacity management.

Errors (4xx / 5xx)

Client and server errors broken out by type — essential for SLA alerting.

Debugging Strategies

- Enable verbose tracing on development aliases; disable for production to reduce overhead.
- Use separate test aliases and environments for staging — never test against prod aliases.
- Instrument token and tool usage per invocation for granular cost visibility.
- Set CloudWatch alarms on error rates, latency spikes, and throttling thresholds.
- Use AgentCore execution traces to refine prompts, instructions, and tool schemas iteratively.

Cost Optimization for Agents

Agent costs accumulate across model invocations, tool calls, Knowledge Base queries, and token consumption. A structured optimization strategy is essential for production workloads.

Model Invocations

Each reasoning and planning step invokes a Bedrock FM. Right-size model choice — use lighter models for routine tasks, frontier models for complex reasoning only.

Token Usage

Keep system and tool prompts concise. Avoid verbosity that inflates input tokens. Use Bedrock prompt caching for repeated prefixes on supported models.

Tool & KB Calls

Batch operations where possible. Cache frequent tool results within the session. Avoid redundant lookups by passing context between steps.

Service Tier Strategy

Priority / Reserved Tier

Use for latency-sensitive, high-SLA production traffic where consistent throughput is critical.

Flex Tier

Use for non-urgent background agent workloads, batch processing, and dev/test environments to significantly reduce cost.

Always monitor token usage per agent invocation alongside tool call counts — these two levers account for the majority of agent cost at scale.

Key Takeaways for Workshop Content

1

Agent Capabilities

Emphasize multi-step reasoning, tool orchestration, and Knowledge Base integration as the core value proposition over simple chatbots.

2

AgentCore Suite

Present Runtime, Gateway, Identity, Memory, Code Interpreter, Browser, and Observability as the production-grade building blocks — not optional extras.

3

Design Patterns

Teach action groups, tool design, and prompt engineering best practices from the very first session. Bad tool design is the most common failure mode.

4

Multi-Agent Architectures

Show when and how to decompose into specialized agents. Demonstrate sequential, parallel, hierarchical, and collaborative patterns with concrete examples.

5

Monitoring & Cost

Stress observability, automated evaluation, and cost tracking as first-class engineering concerns — not afterthoughts. Build these in from day one.